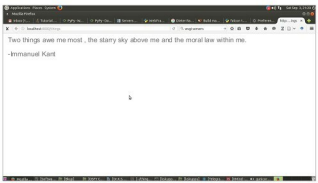

Figure 3: Simple Falcon code -- browser output

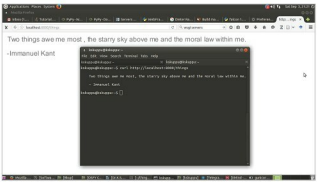

Figure 4: Output at the terminal using Curl

```
# things will handle all requests to the '/things' URL
path
app.add_route('/things', things)
```

To run this code, use the following command:

```
$gunicorn things:app
```

The output can be viewed by directly opening a browser and navigating to the URL, *localhost:8000/things* (as shown in Figure 3).

Otherwise, the output may be viewed through a terminal with the help of Curl, as shown below:

```
$curl localhost:8000/things
```

Understanding Falcon terminology

Falcon terminology is inspired by the REST architectural style. If you already have some exposure to this architectural style then Falcon should feel very familiar to you. But even if you have no clue about REST, etc, it is not difficult to understand Falcon.

Primarily, Falcon maps the incoming resources to entities called 'resources'. Resources are nothing but Python classes, which include a few methods and a specific naming convention. For any HTTP method that you plan your resource to support, you have to add an 'on_x' class method to the resource. The 'x' here indicates any one of the standard HTTP methods listed below:

- on_get
- on_put
- on_head

The methods need to be in lower case. These popular methods are called 'responders'. The responders have two parameters. One indicates the HTTP request and the other indicates the HTTP response to the request.

The request object can be used to read the headers, query parameters and the request body. The following code snippet illustrates response handling:

```
def on_get(self, req, resp):
    resp.data = msgpack.packb({'message': 'Hello world!'})
    resp.content_type = 'application/msgpack'
    resp.status = falcon.HTTP_200
```

The code snippet to handle a POST request and store an image is shown below:

```
import os
import uuid
import mimetypes

import falcon

class Resource(object):

    def __init__(self, storage_path):
        self.storage_path = storage_path

    def on_post(self, req, resp):
        ext = mimetypes.guess_extension(req.content_type)
        filename = '{uuid}{ext}'.format(uuid=uuid.uuid4(),
ext=ext)
        image_path = os.path.join(self.storage_path, filename)

        with open(image_path, 'wb') as image_file:
            while True:
                chunk = req.stream.read(4096)
                if not chunk:
                    break
                image_file.write(chunk)

        resp.status = falcon.HTTP_201
        resp.location = '/images/' + filename
```

After this, fire a POST request as shown below:

```
$ http POST localhost:8000/images Content-Type:image/jpeg <
yourfile.jpg
```

Replace *yourfile.jpg* with the name of the file you want to POST. You may notice that the storage directory that you